AIR (Algebraic Intermediate Representation) 시스템 - 제약 조건 표현 방식 분석

개요

Stwo 프로젝트의 AIR (Algebraic Intermediate Representation) 시스템은 Circle STARK 증명에서 계산 제약 조건을 표현하고 관리하는 핵심 프레임워크입니다. 이 시스템은 복잡한 계산 로직을 수학적 제약 조건으로 변환하여 증명 생성과 검증을 가능하게 합니다.

AIR 시스템의 핵심 구성 요소

1. 제약 조건 표현 시스템 (Constraint Expression System)

1.1 기본 표현식 (BaseExpr)

`BaseExpr` 은 기본 필드 값을 나타내는 표현식으로, 다음과 같은 형태를 가집니다:

```
pub  enum BaseExpr {

Col(ColumnExpr), // 컬럼 참조

Const(BaseField), // 상수 값

Param(String), // 형식 매개변수

Add(Box<BaseExpr>, Box<BaseExpr>), // 덧셈

Sub(Box<BaseExpr>, Box<BaseExpr>), // 뺄셈

Mul(Box<BaseExpr>, Box<BaseExpr>), // 곱셈

Neg(Box<BaseExpr>), // 부호 반전

Inv(Box<BaseExpr>), // 역원
```

```
}
```

주요 특징:

- **컬럼 참조**: `ColumnExpr` 를 통해 특정 상호작용(interaction)의 특정 인덱스와 오프셋을 가진 컬럼을 참조
- **연산 지원**: 기본적인 산술 연산(덧셈, 뺄셈, 곱셈, 부호 반전, 역원)을 지원
- **매개변수화**: 관계의 상호작용 요소와 같은 형식 매개변수를 지원

1.2 확장 표현식 (ExtExpr)

`ExtExpr` 은 보안 필드 값을 나타내는 표현식으로, 4개의 기본 필드 표현식으로 구성됩니다:

```
pub  enum ExtExpr {
SecureCol([Box<BaseExpr>; 4]), // 4개 기본 표현식으로 구성된 보안 컬럼
Const(SecureField), // 보안 필드 상수
Param(String), // 형식 매개변수
Add(Box<ExtExpr>, Box<ExtExpr>), // 덧셈
Sub(Box<ExtExpr>, Box<ExtExpr>), // 뺄셈
Mul(Box<ExtExpr>, Box<ExtExpr>), // 곱셈
Neg(Box<ExtExpr>), // 부호 반전
}
```

2. 컬럼 표현 시스템 (Column Expression System)

2.1 컬럼 식별자 (ColumnExpr)

```
pub struct ColumnExpr {
interaction: usize, // 상호작용 인덱스
idx: usize, // 컬럼 인덱스
offset: isize, // 마스크 오프셋
}
```

역할:

- **상호작용 구분**: 전처리된 트레이스, 원본 트레이스, 상호작용 트레이스를 구분
- **컬럼 인덱싱**: 각 상호작용 내에서 특정 컬럼을 식별
- **시간적 참조**: 오프셋을 통해 현재 행, 이전 행, 다음 행의 값을 참조

2.2 상호작용 유형

```
pub const PREPROCESSED_TRACE_IDX: usize = 0; // 전처리된 트레이스
pub const ORIGINAL_TRACE_IDX: usize = 1; // 원본 트레이스
pub const INTERACTION_TRACE_IDX: usize = 2; // 상호작용 트레이스
```

트레이스 테이블과 제약 조건의 관계

1. 트레이스 테이블 구조

1.1 컴포넌트 트레이스 (ComponentTrace)

```
pub struct ComponentTrace<const N: usize> {
data: [Vec<PackedM31>; N], // N개의 컬럼
```

```
log_size: u32, // 행 수의 로그 값
}
```

특징:

- **SIMD 최적화**: `PackedM31` 을 사용하여 16개 값을 동시에 처리
- **고정 크기**: 컴파일 타임에 컬럼 수가 결정됨
- **병렬 처리**: `par_iter_mut()` 를 통한 병렬 행 반복자 제공

1.2 트레이스 생성 과정

```
// Fibonacci 예제에서의 트레이스 생성
pub fn generate_trace<const N: usize>(
log_size: u32,
inputs: &[FibInput],
) -> ColumnVec<CircleEvaluation<SimdBackend, BaseField, BitReversedOrder>> {
let mut trace = (0..N)
.map(|_| Col::<SimdBackend, BaseField>::zeros(1 << log_size))
.collect_vec();
for (vec_index, input) in inputs.iter().enumerate() {
let mut a = input.a;
let mut b = input.b;
trace[0].data[vec_index] = a;
trace[1].data[vec_index] = b;
trace.iter_mut().skip(2).for_each(|col| {
(a, b) = (b, a.square() + b.square());
```

```
col.data[vec_index] = b;
});
}
// ... 도메인 변환
}
```

2. 제약 조건과 트레이스의 연결

2.1 EvalAtRow 트레이트

```
pub trait EvalAtRow {
type F: FieldExpOps + Clone + Debug + Zero + /* ... */;
type EF: One + Clone + Debug + Zero + /* ... */;
fn next_trace_mask(&mut self) -> Self::F;
fn next_interaction_mask<const N: usize>(
&mut self,
interaction: usize,
offsets: [isize; N],
) -> [Self::F; N];
fn add_constraint<G>(&mut self, constraint: G)
where
Self::EF: Mul<G, Output = Self::EF> + From<G>;
}
```

핵심 메서드:

- `next_trace_mask()` : 다음 트레이스 컬럼의 마스크 값을 반환
- `next_interaction_mask()` : 특정 상호작용의 여러 오프셋에 대한 마스크 값들을 반환
- `add_constraint()` : 제약 조건을 시스템에 추가

2.2 제약 조건 정의 예제

Fibonacci 수열 제약 조건:

```
impl<const N: usize> FrameworkEval for WideFibonacciEval<N> {
fn evaluate<E: EvalAtRow>(&self, mut eval: E) -> E {
let mut a = eval.next_trace_mask();
let mut b = eval.next_trace_mask();
for _ in 2..N {
let c = eval.next_trace_mask();
// 제약 조건: c = a² + b²
eval.add_constraint(c.clone() - (a.square() + b.square()));
a = b;
b = c;
}
eval
}
}
```

Poseidon 해시 제약 조건:

```
pub fn eval_poseidon_constraints<E: EvalAtRow>(
eval: &mut E,
lookup_elements: &PoseidonElements
) {
for _ in 0..N_INSTANCES_PER_ROW {
let mut state: [_; N_STATE] = std::array::from_fn(|_| eval.next_trace_mask());
// 4 full rounds
(0..N_HALF_FULL_ROUNDS).for_each(|round| {
// 라운드 상수 추가
(0..N_STATE).for_each(|i| {
state[i] += EXTERNAL_ROUND_CONSTS[round][i];
});
// 외부 라운드 행렬 적용
apply_external_round_matrix(&mut state);
// S-box 적용 (x^5)
state = std::array::from_fn(|i| pow5(state[i].clone()));
// 각 상태에 대한 제약 조건 추가
state.iter_mut().for_each(|s| {
let m = eval.next_trace_mask();
eval.add_constraint(s.clone() - m.clone());
*s = m;
});
```

```
});

// ... partial rounds 및 나머지 full rounds

}

}
```

다양한 제약 조건 유형과 구현 방식

1. 산술 제약 조건 (Arithmetic Constraints)

1.1 기본 산술 연산

```
// 덧셈 제약 조건

eval.add_constraint(c.clone() - (a.clone() + b.clone()));


// 곱셈 제약 조건

eval.add_constraint(c.clone() - (a.clone() * b.clone()));


// 제곱 제약 조건

eval.add_constraint(c.clone() - a.square());
```

1.2 복합 산술 제약 조건

```
// Fibonacci: c = a² + b²

eval.add_constraint(c.clone() - (a.square() + b.square()));
```

```
// 역원 제약 조건

eval.add_constraint(x0.clone() * intermediate * (x0.clone() + x1.clone()).invers
```

2. 상태 전이 제약 조건 (State Transition Constraints)

2.1 순차적 상태 전이

```
// 이전 행과 현재 행 간의 관계

let [prev_state, curr_state] = eval.next_extension_interaction_mask(

INTERACTION_TRACE_IDX,

[-1, 0]

);

eval.add_constraint(curr_state - prev_state - state_diff);
```

2.2 조건부 상태 전이

```
// 특정 조건에서만 적용되는 제약 조건

let  condition = eval.next_trace_mask();

let  state_change = eval.next_trace_mask();

eval.add_constraint(condition.clone() * (state_change - expected_change));
```

3. 룩업 제약 조건 (Lookup Constraints)

3.1 관계 정의

```
// 매크로를 사용한 관계 정의
relation!(PoseidonElements, N_STATE);


// 관계 사용
eval.add_to_relation(RelationEntry::new(
lookup_elements,
E::EF::one(), // 다중도 (multiplicity)
&initial_state, // 룩업할 값들
));
```

3.2 LogUp 프로토콜

```
impl EvalAtRow for ExprEvaluator {
fn  add_to_relation<R: Relation<Self::F, Self::EF>>(
&mut  self,
entry: RelationEntry<'_, Self::F, Self::EF, R>,
) {
let  frac = Fraction::new(
entry.multiplicity.clone(),
entry.relation.combine(entry.values),
);
self.write_logup_frac(frac);
}
```

```
fn finalize_logup_batched(&mut self, batching: &Batching) {
// 배치된 LogUp 제약 조건들을 최종화
// 누적합 제약 조건 생성
let mut prev_col_cumsum = <Self::EF as Zero>::zero();
for batch_id in (0..last_batch) {
let cur_frac: Fraction<_, _> = fracs_by_batch[&batch_id].iter().cloned().sum();
let [cur_cumsum] = self.next_extension_interaction_mask(
self.logup.interaction,
[0]
);
let diff = cur_cumsum.clone() - prev_col_cumsum.clone();
prev_col_cumsum = cur_cumsum;
self.add_constraint(diff * cur_frac.denominator - cur_frac.numerator);
}
}
}
```

4. 범위 제약 조건 (Range Constraints)

4.1 비트 분해 제약 조건

```
// 값이 특정 비트 수 내에 있음을 보장
fn add_range_constraint<E: EvalAtRow>(eval: &mut E, value: E::F, num_bits: usiz
let mut remaining = value;
```

```
for _ in 0..num_bits {
let bit = eval.next_trace_mask();
// bit는 0 또는 1이어야 함
eval.add_constraint(bit.clone() * (bit.clone() - E::F::one()));
remaining = (remaining - bit.clone()) * E::F::from(BaseField::from_u32_unchecked
}
// 남은 값은 0이어야 함
eval.add_constraint(remaining);
}
```

5. 순열 제약 조건 (Permutation Constraints)

5.1 순열 검증

```
// 두 컬럼이 서로의 순열임을 검증
fn add_permutation_constraint<E: EvalAtRow>(
eval: &mut E,
col1: &[E::F],
col2: &[E::F],
challenge: E::EF,
) {
let prod1 = col1.iter().fold(E::EF::one(), |acc, &val| {
acc * (challenge.clone() - E::EF::from(val))
```

```
});
let prod2 = col2.iter().fold(E::EF::one(), |acc, &val| {
acc * (challenge.clone() - E::EF::from(val))
});
eval.add_constraint(prod1 - prod2);
}
```

제약 조건 최적화 기법

1. 중간값 활용 (Intermediate Values)

```
impl EvalAtRow for ExprEvaluator {
fn add_intermediate(&mut self, expr: Self::F) -> Self::F {
let name = format!("intermediate{}", self.intermediates.len());
let intermediate = BaseExpr::Param(name.clone());
self.intermediates.insert(name.clone(), expr);
self.ordered_intermediates.push(name);
intermediate
}
}

// 사용 예제
let intermediate = eval.add_intermediate(x1.clone() * x2.clone());
```

```
eval.add_constraint(x0.clone() * intermediate * (x0.clone() + x1.clone()).invers
```

2. 배치 처리 (Batching)

```
// 여러 LogUp 항목을 배치로 처리하여 효율성 향상
fn finalize_logup_in_pairs(&mut self) {
let batches = (0..self.logup.fracs.len()).map(|n| n / 2).collect();
self.finalize_logup_batched(&batches)
}
```

3. 차수 경계 최적화 (Degree Bound Optimization)

```
impl ExprEvaluator {
pub fn constraint_degree_bounds(&self) -> Vec<usize> {
let named_exprs = NamedExprs::new(/* ... */);
self.constraints
.iter()
.map(|c| c.degree_bound(&named_exprs))
.collect()
}
}
```

제약 조건 검증 시스템

1. 표현식 평가 (Expression Evaluation)

```
pub trait ExprAssignment {
fn assign(&self, assignment: &ExprVarAssignment) -> SecureField;
}

// 제약 조건 검증
let assignment = eval.random_assignment();
let constraint_values: Vec<SecureField> = eval
.constraints
.iter()
.map(|c| c.assign(&assignment))
.collect();

// 모든 제약 조건이 0이어야 함
assert!(constraint_values.iter().all(|&v| v == SecureField::zero()));
```

2. 다항식 기반 검증

```
pub fn assert_constraints_on_polys<F>(
trace_polys: &TreeVec<ColumnVec<CirclePoly<F>>>,
coset: CanonicCoset,
constraint_eval_fn: impl Fn(AssertEvaluator<'_>),
```

```
claimed_sum: SecureField,
) where
F: FieldExpOps + Clone,
{
// 코셋의 모든 점에서 제약 조건 검증
for point in coset.iter() {
let evaluator = AssertEvaluator::new(/* ... */);
constraint_eval_fn(evaluator);
// 제약 조건이 0인지 확인
}
}
```

결론

Stwo의 AIR 시스템은 다음과 같은 핵심 특징을 가집니다:

1. **표현력**: 복잡한 계산 로직을 수학적 제약 조건으로 표현할 수 있는 풍부한 표현식 시스템
2. **효율성**: SIMD 최적화와 배치 처리를 통한 고성능 제약 조건 처리
3. **모듈성**: 컴포넌트 기반 설계로 재사용 가능한 제약 조건 정의
4. **검증 가능성**: 표현식 평가와 다항식 기반 검증을 통한 제약 조건 정확성 보장

이러한 시스템을 통해 Stwo는 Circle STARK 증명에서 복잡한 계산의 정확성을 효율적으로 증명할 수 있습니다.